

MODULE 17 · PART 5 - RESHAPING

Unnesting and Nested Data

One row into many, many back into one, and the data that arrives already nested.

PART 5 of 8 MODULE 17 of 26 RUNS about 26 min

Why this is the easy half

- ◆ **Nothing here is impossible.** Every operation works on every engine, with a different spelling.
- ◆ **But all of it can lose rows**, silently, and the loss looks exactly like a correct result.
- ◆ **So the difficulty moved.** Module sixteen was about what SQL will not let you do; this one is about what it will.
- ◆ **Which makes the assertion the important part**, rather than the syntax. Count the parents, before and after.

THE TAKEAWAY

A pivot that cannot express your columns fails loudly. An unnest that deletes 4,180 orders does not, and that is the harder failure to live with.

Row count is decided **at runtime**. Column count is decided **at parse time**. That is the whole of Part 5.

The four operations

DIRECTION	WHAT IT IS CALLED
One row into many, from an array	UNNEST, FLATTEN, CROSS JOIN UNNEST
One row into many, from a string	Split into an array first, then unnest
One row into many, from nothing	Cross join to a generated series
Many rows into one	STRING_AGG, LISTAGG, ARRAY_AGG, GROUP_CONCAT

THE TAKEAWAY

Four operations, and the last one has four names because every engine invented it separately. The first three are the same thing with different sources.

Reading an expansion out loud

WRITTEN

SAID

`UNNEST(products)`

one row per element of products

`b, UNNEST(products) AS t(p)`

...and drop baskets whose array is empty

`b LEFT JOIN UNNEST(products) ON true`

...and keep them, with a null element

`str_split(tags, ',')`

turn the string into an array first

`generate_series(1, qty)`

one row per unit, and none if qty is zero

THE TAKEAWAY

Rows two and three are the same operation with different join types, and the second clause of each is the part nobody says out loud.

Unnest, in four dialects

```
1 -- DuckDB, Postgres
2 SELECT txn_id, UNNEST(products) AS product_key FROM basket;
3
4 -- BigQuery, Trino, Presto
5 SELECT txn_id, product_key
6   FROM basket, UNNEST(products) AS product_key;
7
8 -- Snowflake
9 SELECT b.txn_id, f.value::INT AS product_key
10  FROM basket b, LATERAL FLATTEN(input => b.products) f;
11
12 -- Redshift
13 SELECT b.txn_id, p AS product_key FROM basket b, b.products AS p;
```

10 Snowflake returns a row of metadata per element, and value is one column of it. The cast is not optional.

THE TAKEAWAY

Four syntaxes, one operation. Every one of them is a lateral join in disguise, which is why module one's LATERAL slide mattered.

Unnest **fans out and loses rows**. Exactly like an interval join, and for the same reason.

Measured, on 100 baskets

```
1 100 baskets
2   2 with a NULL array
3   3 with an empty array
4 236 elements in total
5
6 UNNEST(products)                -> 236 rows, 95 baskets
7 b, UNNEST(products) AS t(p)     -> 236 rows, 95 baskets
8 b LEFT JOIN UNNEST(products) ON true -> 241 rows, 100 baskets
```

- 6 Five baskets vanish. They had a value and a customer and a store, and they are not in the result.
- 8 The LEFT JOIN brings them back, each with one row and a null element.

THE TAKEAWAY

236 against 241. Five rows, and the only way to see the difference is to count the parents before and after.

Empty and null are different, and both vanish

- ◆ A null array means we do not know what was in it. A basket whose line data failed to load.
- ◆ An empty array means we know there was nothing. An order placed and then fully cancelled.
- ◆ Unnest treats both identically, producing zero rows and deleting the parent, and it distinguishes nothing.
- ◆ LEFT JOIN keeps the parent, and then a null element means either. Carry a flag if the difference matters.

THE TAKEAWAY

Module two's distinction, one level up. Absent and unknown are different claims, and an array collapses them into the same zero rows.

Five rows, and the answers by hand

```

1 id  products      UNNEST gives
2 --  -----
3 1  [10, 20, 30]    3 rows
4 2  [40]           1 row
5 3  []             0 rows  <- id 3 vanishes
6 4  NULL           0 rows  <- id 4 vanishes
7 5  [50, NULL]     2 rows, one with a null element
8
9 -- 5 rows in, 6 rows out, 3 parents surviving

```

7 A null INSIDE the array still produces a row. A null array does not. Those are different things.

THE TAKEAWAY

Five rows covering every case: several elements, one, empty, null, and a null element. Every defect in this module is visible in this fixture.

Testing an expansion

- ◆ **A row with an empty container**, which is the case that silently deletes a parent.
- ◆ **A row with a null container**, which does the same thing for a different reason.
- ◆ **A container holding a null**, which does not delete anything and looks like the other two in the output.
- ◆ **And the parent count asserted**, because the first two are invisible in every other way.

THE TAKEAWAY

Three test rows and one assertion. Production data rarely contains an empty array at the moment you happen to be testing, and always contains one eventually.

An order-line report unnests a JSON array of items held on `fact_order`, producing one row per order line. It is used to reconcile order value against the line total.

The order count in the report is 4,180 lower than the order count in `fact_order`. Every order in the report has correct lines, and no error is raised. The missing orders all have a value and a customer.

What is the cause?

- ☐ A The arrays on those orders contain nulls, which are being filtered out by the unnest.
- ☐ B The JSON on those rows failed to parse, so the array came back as an error value and was skipped.
- ☐ C UNNEST produces zero rows for an empty or null array, so the parent order disappears entirely.
- ☐ D The count is correct. An order with no lines is not an order and should not be in a line report.

◆ PAUSE AND DECIDE

What is the cause?

A Nulls inside the arrays

A null element still produces a row. It is a null array, or an empty one, that produces no rows at all.

C Empty or null arrays

Zero elements means zero rows, so the parent is deleted by the implicit inner join. LEFT JOIN UNNEST keeps it.

B A parse failure

Most engines raise on malformed JSON rather than skipping the row. And a parse error would not correlate with orders having a value.

D The count is correct

Defensible for a line report and not for a reconciliation. These orders have a value that now appears on neither side.

THE SIGNAL

"The missing orders all have a value and a customer." They are real orders with no lines recorded, which is a data question. Deleting them silently is not the answer to it.

Unnest with position

```
1 -- Postgres
2 SELECT id, e, ord
3   FROM t, UNNEST(products) WITH ORDINALITY AS u(e, ord);
4
5 -- DuckDB
6 SELECT id, UNNEST(products) AS e,
7        generate_subscripts(products, 1) AS ord FROM t;
8
9 -- Snowflake: FLATTEN returns INDEX as one of its columns
10 SELECT t.id, f.value AS e, f.index AS ord
11   FROM t, LATERAL FLATTEN(input => t.products) f;
```

3 WITH ORDINALITY gives the element's position. Without it, the array's order is not recoverable from the rows.

THE TAKEAWAY

An array is ordered and a set of rows is not. Unnesting without capturing the position throws that ordering away, and nothing later can reconstruct it.

When the order in the array matters

- ◆ **Line numbers on a receipt.** The array holds them in order, and the rows will not unless you say so.
- ◆ **Re-aggregating without the position scrambles it,** so the round trip returns a different array.
- ◆ **Capture ordinality on the way out, order by it on the way back.** Then the round trip is faithful.
- ◆ **And if the order never mattered,** say so, because an unordered array is a set stored in a list.

THE TAKEAWAY

The round trip is only an identity when you carried the position. Without it, nest, unnest and nest again returns the same elements in an arbitrary order.

Aggregating rows into one value. Easy, and it has **an ordering problem**.

Aggregating, in four dialects

```
1 -- Postgres, DuckDB
2 STRING_AGG(product_key::TEXT, ',' ORDER BY line_no)
3 ARRAY_AGG(product_key ORDER BY line_no)
4
5 -- Redshift, Oracle, Snowflake, Db2
6 LISTAGG(product_key, ',') WITHIN GROUP (ORDER BY line_no)
7
8 -- SQL Server 2017+
9 STRING_AGG(product_key, ',') WITHIN GROUP (ORDER BY line_no)
10
11 -- MySQL
12 GROUP_CONCAT(product_key ORDER BY line_no SEPARATOR ',')
```

6 **WITHIN GROUP** again, from module nine. The same clause that ordered a percentile orders a concatenation.

THE TAKEAWAY

Four names, four syntaxes, one operation. The **ORDER BY** inside is optional in every one of them and should be treated as compulsory.

Three things that bite here

- ◆ **No ORDER BY means no guarantee**, and on a single node it will look stable for years. Module two, again.
- ◆ **LISTAGG has a length limit**. Oracle overflows at 4,000 characters and raises; Redshift caps at 65,535.
- ◆ **And nulls are skipped, not rendered**. Three rows with a null in the middle give two elements, not three.
- ◆ **So coalesce before aggregating** if position matters, because otherwise the string is shorter than the group.

THE TAKEAWAY

The length limit is the one that fails loudly and at the worst moment, because it depends on how many rows are in the group rather than on anything in the code.

The round trip loses rows too

```
1 100 baskets -> UNNEST -> 236 rows -> LIST() -> 95 baskets
2
3 -- the five with empty or null arrays did not come back,
4 -- because they never produced a row to aggregate.
```

1 Measured. Exactly the same asymmetry as module sixteen's pivot and unpivot round trip, for exactly the same reason.

THE TAKEAWAY

Nest, unnest, nest again is not an identity. Neither is pivot, unpivot, pivot. Both lose the rows that had nothing in them.

A comma-separated list in a column is **a modelling failure** you have inherited.

An array column is **a denormalisation**.
Sometimes a good one, and always a
decision.

When an array column earns its place

- ◆ **The elements are always read with the parent**, and never on their own. A list of tags shown on a page.
- ◆ **The cardinality is small and bounded**, so a basket of four lines rather than an order of four thousand.
- ◆ **Not when anything filters or aggregates on them**, because that is an unnest on every query.
- ◆ **And a bridge table is the alternative**, which is one row per parent per element, stored once rather than derived.

THE TAKEAWAY

The same trade as module fifteen's periods and events. Store the shape the questions want, and derive the other one where it is needed.

Splitting, and what it does to edges

```
1 id s
2 1 'a,b,,c' -> ['a','b','','c'] 4 elements
3 2 '' -> [''] 1 element, an empty string
4 3 NULL -> NULL 0 elements, row lost
5
6 -- three rows in, five rows out of the unnest
7 -- and one of those five is an empty string that means nothing
```

- 3 An empty string splits into one empty element, not zero. That row produces a tag that does not exist.
- 4 And a null string produces nothing, so the row disappears, exactly as with an array.

THE TAKEAWAY

Measured. Every edge case in a delimited string is a different kind of nothing, and splitting turns them into three different results.

Why you cannot just use LIKE

- ◆ **LIKE '%promo%' matches inside other tags.** It finds 'promotional' and 'nonpromo' as well as 'promo'.
- ◆ **Delimiting both ends fixes that**, searching for 'promo,' in '||tags||', and it is unreadable and unindexable.
- ◆ **And it cannot answer anything aggregate.** How many tags, which tags are most common, which co-occur.
- ◆ **Split at load, into a bridge table.** One row per entity per tag, and every question becomes an ordinary join.

THE TAKEAWAY

Module twelve's delimiter lesson, arriving again. Searching a delimited string is a substring match pretending to be a set membership test.

A merchandising feed delivers product attributes as a single comma-separated string per product. Analysts need to filter on individual tags, count products per tag, and find tags that co-occur.

The source system is owned by another team and will not change this year. The current approach is a LIKE pattern per tag, and a report has just miscounted because 'promo' matched 'promotional'.

What should they do?

- ☐ A Keep LIKE, but pad the column and the search term with delimiters so that partial matches cannot occur.
- ☐ B Split and unnest at load into a bridge table of one row per product per tag, and query that.
- ☐ C Ask the source team to deliver the attributes as JSON instead, which is a structured format.
- ☐ D Use a regular expression with word boundaries, which handles the partial-match problem precisely.

◆ PAUSE AND DECIDE

What should they do?

A Pad and LIKE

Fixes the partial match and nothing else. Counting products per tag and finding co-occurrence are still impossible.

C Ask for JSON

The source will not change this year, and a JSON array of strings needs unnesting too. It moves the split, not removes it.

B Split at load

One split, once, and filtering, counting and co-occurrence all become ordinary joins and group bys.

D A regex

Precise, slower than LIKE, and it answers only the filtering question. The aggregate questions need rows, not matches.

THE SIGNAL

"Count products per tag, and find tags that co-occur." Both of those need one row per tag. No string operation produces rows, and that is the whole argument.

The third direction. **Generating rows** that no table contains.

Expanding one row into a range

```
1 -- one row per month of a contract's term
2 SELECT c.contract_id,
3        (c.start_date + INTERVAL (n) MONTH)::DATE AS period
4 FROM dim_contract c,
5      generate_series(0, c.term_months - 1) AS t(n);
6
7 -- one row per unit sold, from a quantity
8 SELECT s.sale_key, s.product_key
9 FROM fact_sales_line s,
10    generate_series(1, s.qty) AS t(n);
```

- 4 A lateral join: the series bound depends on a column of the row. Module one's exception, doing something useful.
- 9 A quantity of zero or null produces no rows, and the sale disappears. The same trap, for the third time.

THE TAKEAWAY

Function names differ: `generate_series` on Postgres and DuckDB, `GENERATE_ARRAY` on BigQuery, `SEQUENCE` on Snowflake and Trino, and a numbers table everywhere else.

The same trap, three times

- ◆ An empty array produces no rows, so the parent vanishes.
- ◆ A null string produces no rows, so the parent vanishes.
- ◆ A zero or null quantity produces no rows, so the parent vanishes.
- ◆ All three are an implicit inner join. LEFT JOIN, or a COALESCE to a one-element default, fixes all three.

THE TAKEAWAY

Every expansion in SQL is a join, and every join has a type. The default is inner, and inner deletes the rows that expanded to nothing.

JSON in a column. **Schema on read**, and the reading is where the cost lands.

Every expansion is a join, and **every join has a type**. The default deletes what expanded to nothing.

Extraction, and its two operators

```
1 doc = {"tier":"Gold", "spend":142.50, "tags":["a","b"], "opt":null}
2
3 doc -> 'tier'      = "Gold"      -- JSON, quotes included
4 doc ->> 'tier'     = Gold        -- text
5 doc ->> 'spend'    = '142.5'    -- text, not a number
6 typeof(...)      = VARCHAR
7
8 doc ->> 'missing'  = NULL
9 doc ->> 'opt'      = NULL        -- indistinguishable
```

- 5 Everything comes out as text. A numeric comparison needs a cast, and the cast is yours to remember.
- 9 An absent key and a JSON null both give SQL NULL. Nothing in the result tells you which.

THE TAKEAWAY

Measured. The last two lines are the whole difficulty of JSON in SQL: the type is gone, and three different kinds of nothing arrive as one null.

Three kinds of nothing

- ◆ **The key is absent.** This document does not record a tier, perhaps because it predates the field.
- ◆ **The key is present and null.** The system knows about tier and recorded no value for it.
- ◆ **The extraction returned SQL NULL.** Which of the two above, and you cannot tell from the value.
- ◆ **Test for the key separately,** with a key-existence operator or function, if the difference means anything.

THE TAKEAWAY

Module two said absent and unknown are different claims. JSON adds a third: the field did not exist when this row was written, which is a schema fact rather than a data one.

A membership report counts members whose loyalty tier is not set. It extracts the tier key from a JSON profile column and counts where the result is null. It reports 84,000 members with no tier.

The loyalty team say the true figure is around 11,000. The other 73,000 are members whose profile was written before the tier field existed at all. Both counts are reproducible.

What is the cause?

- A The comparison should use IS NOT NULL rather than IS NULL, so the logic is inverted.
- B The extracted value needs a cast to text before it can be compared to null correctly.
- C The JSON on the older rows is malformed, so extraction fails and returns null for every key.
- D Extraction returns SQL NULL for an absent key and for a JSON null alike. A key-existence test separates them.

◆ PAUSE AND DECIDE

What is the cause?

A Inverted logic

Would give the complement, around 1.1 million. The reported figure is far too small for that and far too large to be right.

C Malformed JSON

Would fail on every key rather than one, and most engines raise on malformed JSON rather than returning null.

B A missing cast

Casting matters for comparing values and not for testing nullness. NULL is NULL whatever type it was going to be.

D Two nothings, one NULL

73,000 documents have no tier key and 11,000 have a null one. Both extract to NULL, and only key existence separates them.

THE SIGNAL

"Written before the tier field existed." That is a schema fact about the document, not a data fact about the member. SQL's NULL cannot carry the difference.

JSON arrays of objects

```
1 -- items: [{"sku":101,"qty":2},{"sku":205,"qty":1}]
2
3 SELECT o.order_id,
4         (item ->> 'sku')::INT AS sku,
5         (item ->> 'qty')::INT AS qty
6 FROM fact_order o,
7         UNNEST(json_extract(o.items, '$[*]')) AS t(item);
```

- 7 Unnest the array first, then extract from each element. Two operations, and both can lose rows.
- 4 And both casts are yours. An engine that stored the number as JSON hands it back as text.

THE TAKEAWAY

The commonest real shape: an array of objects on a fact row. It needs an unnest and two extractions, and every one of those is a place a row or a type can go missing.

Nested objects and the path

- ◆ **A path reaches into nesting.** Dollar dot address dot postcode, rather than one extraction per level.
- ◆ **A missing level gives null, not an error,** so a typo in a path is indistinguishable from an absent field.
- ◆ **Which is why a path typo can survive for months.** The column is null, and null is a plausible value.
- ◆ **Assert that a required path is non-null on some rows,** because a path that is null everywhere is almost certainly wrong.

THE TAKEAWAY

The strongest JSON assertion there is: this field should be present on at least some rows. A path that returns null on every row has never been checked by anything.

What JSON costs on every read

- ◆ **Parsing, on every row, on every query**, unless the engine stores a pre-parsed binary form as Postgres jsonb does.
- ◆ **A cast, on every extraction**, which is yours to write and yours to get wrong.
- ◆ **No statistics**, so the planner has no idea how selective a predicate on an extracted key is.
- ◆ **And no constraint**. Nothing enforces that tier is one of four values, or that it is present at all.

THE TAKEAWAY

The last two are the expensive ones. A column the planner cannot estimate and the schema cannot constrain is a column every query has to be careful about.

Promoting the keys you actually use

```
1 CREATE OR REPLACE TABLE dim_member AS
2 SELECT member_durable_key,
3         profile ->> 'tier' AS loyalty_tier,
4         (profile ->> 'spend')::NUMERIC(12,2) AS annual_spend,
5         json_exists(profile, '$.tier') AS tier_key_present,
6         profile AS profile_json
7 FROM stg_member;
```

- 5 The key-existence flag, computed once at load, so nobody downstream has to know it matters.
- 6 And the blob is kept, so a key nobody promoted yet is still reachable.

THE TAKEAWAY

Typed, statistically visible, constrainable columns for the keys that get queried, and the document kept for the rest. The flexibility is not lost, just not paid for on every read.

Kestrel stores member profiles as a JSON column on a 7-million-row table. Eleven reports extract and cast the same three keys on every run, and each defines the casts slightly differently.

Queries are slow, and the planner consistently underestimates rows for predicates on the extracted tier. A new report has just failed because it cast spend as an integer.

What is the right change?

- A Extract the three hot keys into typed columns at load, keep the document for the rest, and have the reports read the columns.
- B Add a GIN index on the JSON column, which lets the planner use the extracted predicates efficiently.
- C Move the extraction and casting into the BI layer, where each report can define the types it needs.
- D Store all member attributes as JSON, so that adding a new attribute never requires a schema change.

◆ PAUSE AND DECIDE

What is the right change?

A Promote the hot keys

One cast, one definition, real statistics, and eleven reports stop disagreeing about what spend is.

C Cast in the BI layer

Moves eleven inconsistent definitions somewhere with less review. The integer cast would have failed there too.

B A GIN index

Helps containment lookups on Postgres, does nothing for the per-row cast, and does not exist on Redshift at all.

D Everything as JSON

Doubles down on the thing that caused this. Schema changes are the cost; no statistics and no constraints are the price.

THE SIGNAL

"Each defines the casts slightly differently." Eleven definitions of one field is a modelling problem, and no index or engine setting fixes a modelling problem.

The assertion for an expansion

```
1 SELECT
2     (SELECT COUNT(*) FROM fact_order)           AS parents_in,
3     (SELECT COUNT(DISTINCT order_id) FROM order_line) AS parents_out,
4     (SELECT COUNT(*) FROM fact_order
5      WHERE items IS NULL OR json_array_length(items) = 0) AS empty_or_null,
6     (SELECT COUNT(*) FROM order_line WHERE sku IS NULL) AS unparsed_skus;
```

3 These two must differ by exactly the count on line 5, and by nothing else.

THE TAKEAWAY

Three numbers with an arithmetic relationship between them. If parents in minus parents out does not equal the empty count, something else is being lost.

What Part 5 gave you

- ◆ **Module sixteen: values into names**, which is one-way, and which needs the column list before it runs.
- ◆ **Module seventeen: rows into rows**, which is free, and which quietly deletes anything that expanded to nothing.
- ◆ **One asymmetry explains both.** Runtime row count, parse-time column count.
- ◆ **And two assertions cover the part.** Pivoted total against long total, and parent count before against after.

THE TAKEAWAY

Two modules, two assertions, one line each. Part 5 is the shortest part of the course and it has the highest ratio of silent failures to syntax.

Semi-structured, by engine

ENGINE	WHAT IT GIVES YOU
PostgreSQL	json and jsonb. jsonb is pre-parsed, indexable with GIN, and the one to use.
Snowflake, BigQuery	Native VARIANT or JSON types, with path access and FLATTEN.
Amazon Redshift	SUPER, with PartiQL path syntax. No GIN-style index.
DuckDB	A JSON extension, loaded by default, with the Postgres operators.
SQL Server	JSON stored as text, with OPENJSON and JSON_VALUE to read it.

THE TAKEAWAY

Five engines and five storage models. The path syntax is broadly similar everywhere; the storage, the indexing and the performance are not.

Deciding, in four questions

01

Direction

Expanding or collapsing? Expanding is a join with a type; collapsing is an aggregate with an ordering.

02

Empties

What happens to a row whose array, string or quantity is empty or null? Decide, do not inherit.

03

Once or every time

Split and extract at load, into real columns and real rows. Not in every query that needs them.

04

Assert

Parent count before and after. One comparison, and it catches every loss in this module.

THE TAKEAWAY

Step four is the same assertion as module fifteen's, because the failure is the same failure. An expansion is a join, and an inner join deletes what expanded to nothing.

The confusions, settled

SYMPTOM	THE CAUSE
Fewer parents after unnesting	Empty or null arrays. The implicit join is inner.
A tag that is an empty string	Splitting " gives one empty element, not zero.
A concatenated list in a different order each run	No ORDER BY inside the aggregate.
LISTAGG fails on a large group	A length limit. 4,000 on Oracle, 65,535 on Redshift.
A JSON number compares wrongly	Extraction returns text. The cast is yours to write.
Too many rows with no value for a key	Absent key and JSON null both extract to SQL NULL.

THE TAKEAWAY

Rows one and six are the same lesson at two levels. A structure that contains nothing and a structure that is not there produce identical results, and mean different things.

Five habits

- ◆ **Count the parents before and after any expansion.** One comparison, and it catches every loss in this module.
- ◆ **LEFT JOIN when unnesting,** unless you can say why a row with nothing in it should be deleted.
- ◆ **Always write the ORDER BY inside an aggregate,** because a set has no order and single-node stability is not a guarantee.
- ◆ **Split delimited strings at load,** into a bridge table, and never write a LIKE against a list.
- ◆ **Promote the JSON keys you query** into typed columns, and keep the document for everything else.

THE TAKEAWAY

The first two are one line each and would have prevented three of this module's four scenarios. The last two are modelling decisions that prevent the other one.

Part 6 is about combining results rather than reshaping them.